Практика №2. Аудит готового клиент-серверного приложения

ФИО: Асадчий Константин Михайлович

Группа: 23ис-41

1. Ответы на вопросы

1. Какой протокол используется в send_reliable?

Используется TCP: сокет создаётся с типом SOCK_STREAM, а connect() устанавливает соединение с сервером. TCP передаёт поток байтов по порядку и обрабатывает повторную отправку потерянных данных; при разрыве связи операция может завершиться ошибкой.

2. Какой протокол используется в send_fast?

Используется UDP: сокет имеет тип SOCK_DGRAM, а сообщение отправляется через sendto(). Устанавливать соединение перед отправкой не нужно, но доставка сообщения и получение ответа не гарантируются.

3. В каком формате передаются данные?

Клиент передаёт данные в JSON, а перед отправкой кодирует текст в UTF-8. Это подтверждает выражение json.dumps(message, ensure_ascii=False).encode("utf-8"), которое используется в обеих функциях клиента.

4. Что произойдёт при отправке hello?

Если отправить именно байты b"hello", сервер не сможет разобрать их как JSON и попадёт в обработчик JSONDecodeError. Он запишет ошибку в консоль и вернёт {"status": "ERROR", "reason": "invalid JSON"}; TCP-соединение после этого закроется, а сервер продолжит ждать клиентов.

Если передать строку "hello" через json.dumps(), получится корректная JSON-строка, которую этот сервер примет. Поэтому для проверки неверного формата отправлялись сырые байты b"hello".

2. Запуск сервера и клиента

Исходные server.py, client.py и format_check.py взяты из задания. Сервер и клиент запущены отдельными процессами Python 3.12 на одном компьютере, адрес localhost. Код приложения не исправлялся.

Сохранённый вывод сервера

```
python -u server.py
[TCP] Сервер запущен на localhost:9000
[UDP] Сервер запущен на localhost:9001
[TCP] Получено от ('127.0.0.1', 37534): {'city': 'Москва', 'temp': -5, 'humidity': 80,
  'is_snowing': True}
[UDP] Получено от ('127.0.0.1', 47634): {'city': 'Москва', 'temp': -5, 'humidity': 80,
  'is_snowing': True}
```

Сервер открыл порты 9000 для TCP и 9001 для UDP. В журнале появились два одинаковых объекта с погодой, полученные разными способами. Порты клиента назначались системой автоматически.

Сохранённый вывод клиента

```
python client.py
=== Отправка через надёжный протокол ===
Ответ: {'status': 'OK', 'protocol': 'TCP', 'format': 'JSON'}
Время: 0.44 мс

=== Отправка через быстрый протокол ===
Ответ: {'status': 'OK', 'protocol': 'UDP', 'format': 'JSON'}
Время: 0.73 мс
```

Оба запроса получили статус OK. В этом запуске TCP занял 0,44 мс, UDP - 0,73 мс. Это единичные измерения на localhost: они не доказывают, что один протокол всегда быстрее другого. У TCP отсчёт в программе начинается уже после connect(), поэтому время установления соединения не учитывается.

Вместо снимков двух окон приведены журналы реального запуска: в используемой среде нет доступных окон терминала для снимка экрана. Текст ответов и измеренное время взяты из вывода программ.

3. Тестирование граничных случаев

Случай 3.1. Сервер выключен, TCP

Остановил сервер и снова запустил client.py. На connect() появилась ошибка ConnectionRefusedError: [Errno 111] Connection refused. На localhost:9000 никто не принимал соединение. Ошибка не обрабатывается, поэтому клиент завершился и до UDP-части не дошёл.

```
ConnectionRefusedError: [Errno 111] Connection refused
```

Случай 3.2. Сервер выключен, UDP

При выключенном сервере вызвал только send_fast(weather) из исходного модуля client.py. Функция вернула статус TIMEOUT примерно через 2,001 секунды. Ответа не было, а ожидание завершилось по установленному client.settimeout(2.0).

```
python -c 'from client import send_fast,weather; print(send_fast(weather))'
({'status': 'TIMEOUT'}, 2.0007373740002095)
```

Случай 3.3. Отправка не JSON

Запустил сервер и отдельным TCP-запросом отправил b"hello". Это те же байты, которые предлагается временно подставить в client.py. Исходный файл клиента при этом менять не пришлось.

```
Ответ: {"status": "ERROR", "reason": "invalid JSON"}
Сервер: [TCP] Ошибка: не удалось разобрать JSON от клиента
```

Сервер сообщил об ошибке формата и отправил ERROR. Исключение JSONDecodeError обработано, поэтому неверный запрос не остановил сервер.

Случай 3.4. Сравнение форматов

Запустил format_check.py. JSON занял 72 байта, XML - 111 байт. Для этого объекта JSON меньше на 39 байт, то есть примерно на 35,1% относительно XML. В XML названия полей записываются в открывающих и закрывающих тегах, поэтому сообщение длиннее.

3. Тестирование - сравнение восстановленных данных

```
JSON: {"city": "Москва", "temp": -5, "humidity": 80, "is_snowing": true}
Размер JSON: 72 байт
XML: <weather><city>Москва</city><temp>-
  5</temp><humidity>80</humidity><is_snowing>True</is_snowing></weather>
Размер XML: 111 байт

Восстановлено из JSON: {'city': 'Москва', 'temp': -5, 'humidity': 80, 'is_snowing': True}
Восстановлено из XML: {'city': 'Москва', 'temp': '-5', 'humidity': '80', 'is_snowing':
  'True'}
Данные совпадают: True
```

Из JSON восстановился исходный словарь с числом -5, числом 80 и логическим значением True. В restored_xml все значения стали строками: "-5", "80" и "True". Для полного совпадения с исходным словарём эти значения нужно преобразовать в нужные типы.

Надпись «Данные совпадают: True» проверяет только restored_json == weather. XML с исходными данными в этой строке не сравнивается. Кроме того, restored_xml строится из уже существующего root, а не путём разбора xml_data, поэтому обратное чтение XML-текста отдельно не проверяется.

4. Вывод

Для этого приложения я выбрал бы TCP и JSON. Здесь важно получить ответ сервера, а сообщение маленькое, поэтому удобнее использовать соединение и надёжную передачу. JSON в проведённой проверке компактнее XML и сохраняет типы значений без отдельного преобразования.

UDP можно использовать, если потеря отдельных обновлений погоды допустима. Но в данном коде нет повторной отправки запроса, поэтому после потери сообщения или ответа клиент получит только TIMEOUT.

Что стоит исправить в программе

1. Обрабатывать ошибку подключения TCP и установить таймаут ожидания. Сейчас ошибка завершает клиент, а ожидание ответа при зависшем сервере может затянуться.

2. Заменить send() на sendall() и определить границы TCP-сообщения, например передавать его длину. Один recv(4096) не гарантирует получение всего JSON: TCP передаёт поток, а не отдельные сообщения.

3. Обрабатывать неверную кодировку UTF-8 и сетевые ошибки. Сейчас перехватывается только JSONDecodeError, а UnicodeDecodeError может остановить TCP-поток или UDP-сервер.

4. Проверять поля объекта погоды и их типы. Сейчас сервер подтверждает любой корректный JSON, даже если в нём нет city или temp.

Эти предложения сделаны по коду. В работе выполнены обычный запуск и четыре проверки из задания; отдельные проверки всех перечисленных недостатков не проводились.